DSC 202 Final Report: Project BridgeWell

Employee Support Solutions

Diana Freeman, Sruthi Papanasa, Mitra Rezvany, Natalie Tehranchi
March 16, 2026
GitHub Repo: https://github.com/n-tehranchi/Project-BridgeWell

Introduction

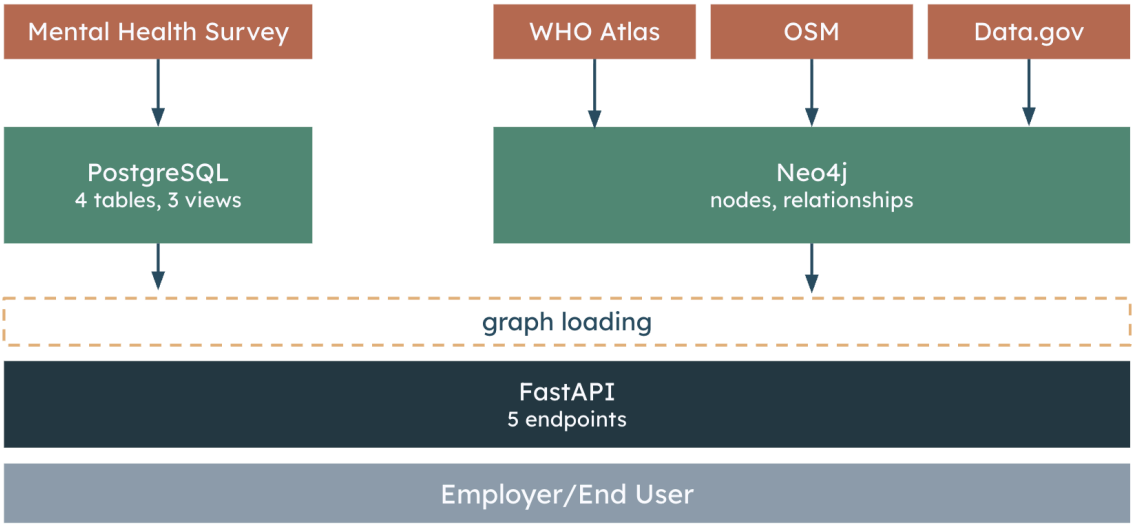
Mental health in the workplace is a growing global concern. Based on survey responses, approximately 75% of workers across every country lack mental health support at work. Employees face rising levels of stress, burnout, and untreated mental health conditions, yet many organizations lack the tools to identify and address these needs effectively.

BridgeWell is a data-driven platform designed to bridge this gap. Given a company profile, including country, industry, size, and remote work policy, BridgeWell analyzes the mental health landscape of that country using real survey data and WHO indicators, then returns targeted wellbeing program recommendations.

The system is built on a dual-database architecture: PostgreSQL handles structured relational analytics over a 10,000-respondent mental health survey, while Neo4j powers a knowledge graph linking countries to identified mental health needs and wellbeing programs. A FastAPI layer exposes this functionality through six RESTful endpoints.

System Architecture

BridgeWell's architecture has four layers that transform raw data to employer-facing recommendations.



The four data sources feed into PostgreSQL, which drives the graph loading pipeline into Neo4j. The FastAPI layer sits on top of both databases, combining their outputs into a single response for the end user.

The Data

The primary data source is a mental health survey dataset containing 10,000 respondents across multiple countries. Each record captures demographic background, workplace conditions, lifestyle factors, and detailed symptom scoring. The raw CSV is processed and normalized into four PostgreSQL tables during the data loading phase.

Three additional external sources populate the Neo4j knowledge graph:

- WHO Atlas: psychiatrist, density, mental health budget allocation, and national policy indicators by country
- OpenStreetMap (OSM) Overpass API: geolocated mental health facilities by country
- data.gov DCAT API: government-published wellness program datasets used as supporting evidence for program recommendations

PostgreSQL

PostgreSQL serves as the relational backbone of BridgeWell, storing the survey dataset in a normalized four-table schema and exposing aggregate country-level statistics through three SQL views consumed directly by the API.

Schema Design

The schema is organized around a central respondents table that links to three child tables, each joined by a foreign key on respondent_id. This normalization avoids repeating demographic data across records and makes it easy to query any combination of the four domains independently.

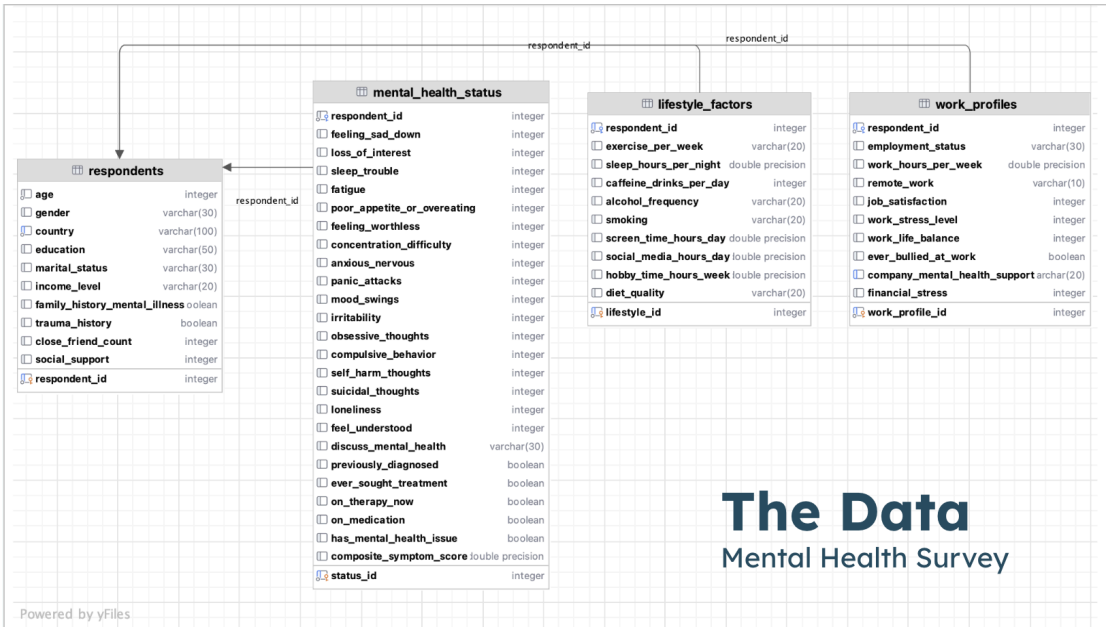


Table	Rows	Key Fields
respondents	1 per participant	age, gender, country, education, income_level, social_support
work_profiles	1 per respondent	work_stress_level, job_satisfaction, work_life_balance, company_mental_health_support
lifestyle_factors	1 per respondent	exercise_per_week, sleep_hours_per_night, diet_quality, alcohol_frequency
mental_health_status	1 per respondent	15 symptom scores, diagnosis flags, composite_symptom_score (computed)

A notable design choice is the composite_symptom_score column in mental_health_status, defined as a GENERATED ALWAYS AS STORED column in PostgreSQL. It averages 15 individual symptom scores at write time, so the API reads a single pre-computed value rather than re-aggregating on every query.

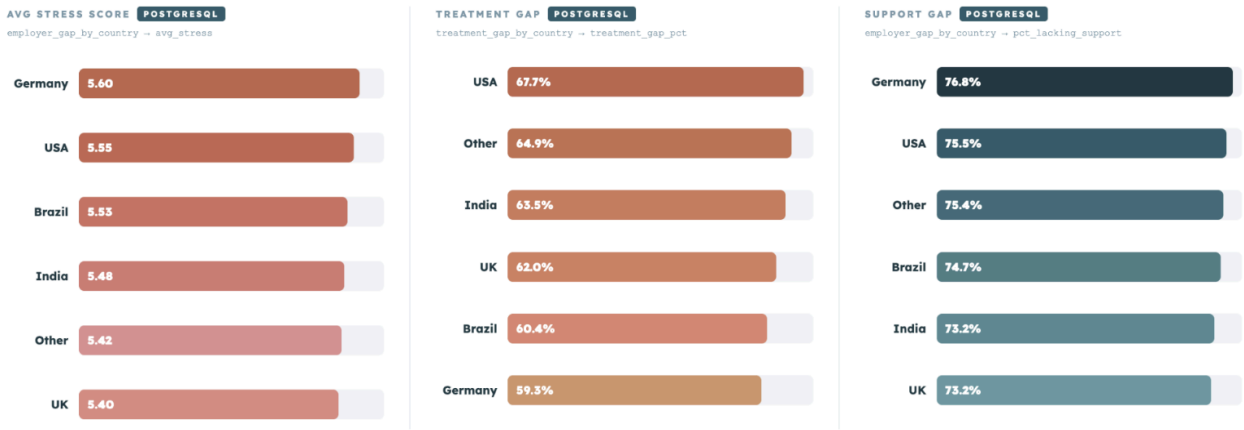
SQL Views

Three views pre-aggregate the survey data at the country level and serve as the primary interface between PostgreSQL and the API layer.

View	Purpose
employer_gap_by_country	Average stress, work-life balance, symptom score, and % lacking mental health support - ordered by severity
lifestyle_risk_by_country	Average sleep, screen time, caffeine intake, and counts of respondents who never exercise, smoke, or drink frequently
treatment_gap_by_country	Diagnosed count, untreated count, and % untreated per country

Exploring the Data

Querying the three views reveals significant mental health disparities across countries. Key findings from the survey data:



Metric	Highest	Lowest
Avg Stress Score	Germany: 5.60 / 10	UK: 5.40 / 10
Treatment Gap %	USA: 67.7% untreated	Germany: 59.3% untreated
Support Gap %	Germany: 76.8% lacking support	UK / India: 73.2%

These findings directly motivate the recommendation engine: countries with high stress, high treatment gaps, and low employer support generate more urgent and varied need nodes in the graph.

Neo4j

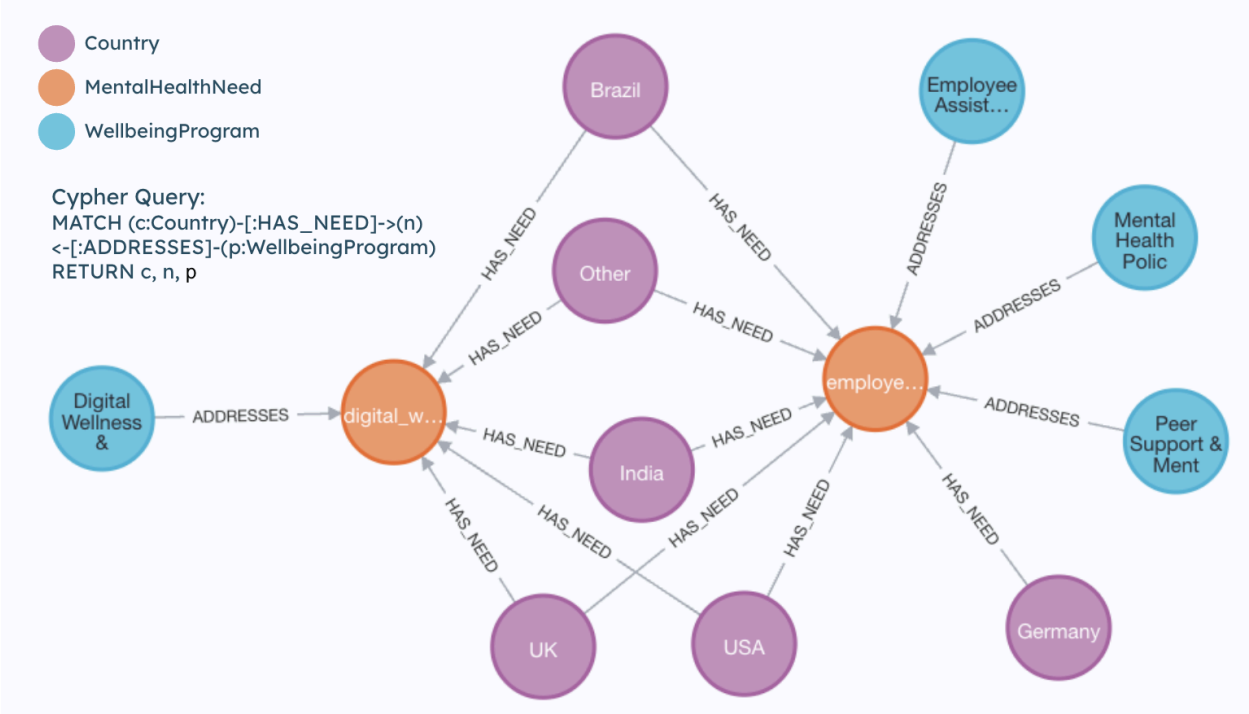
Neo4j stores BridgeWell's knowledge graph, which models relationships between countries, mental health needs, wellbeing programs, WHO data, healthcare facilities, and government datasets. The graph layer enables the recommendation engine. Rather than querying a flat table of programs, the API traverses relationship paths to find programs that specifically address the identified needs of a given country.

Graph Schema

The graph is built around the Country node as a central hub. Seven node types are defined, each with uniqueness constraints enforced at the database level:

Node Label	Description
Country	Central hub - one node per country
SurveySnapshot	Aggregated survey metrics for a country (stress, symptom score, treatment gap)

WHOProfile	WHO Atlas data: psychiatrists per 100k, MH budget %, national policy flag
MentalHealthNeed	A specific need category (e.g. Stress Management, Anxiety & Depression) with controlled vocabulary
WellbeingProgram	A wellbeing program with type, description, and remote availability flag
Facility	A physical mental health facility (hospital, clinic, community center)
GovDataset	A government-published wellness dataset from data.gov, linked to programs as evidence

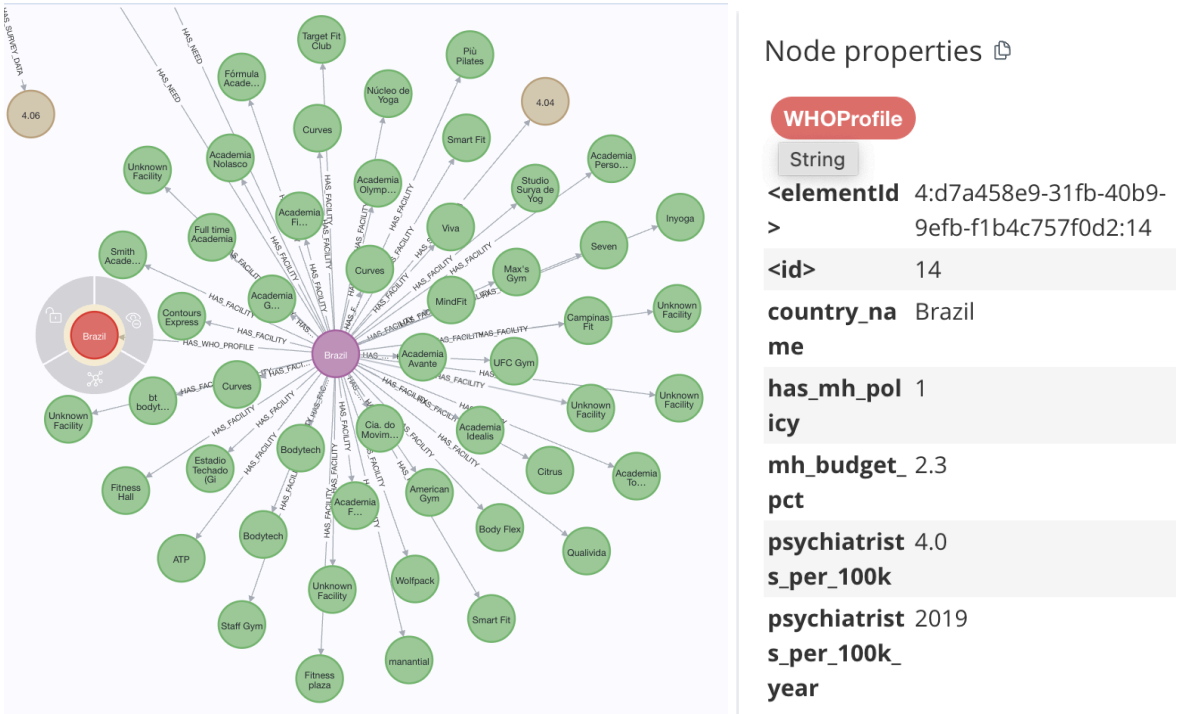


The key relationship that drives recommendations is the HAS_NEED edge between Country and MentalHealthNeed, which carries a severity_score property. The ADDRESSES relationship connects MentalHealthNeed nodes to WellbeingProgram nodes, forming the traversal path used by the recommendations endpoint.

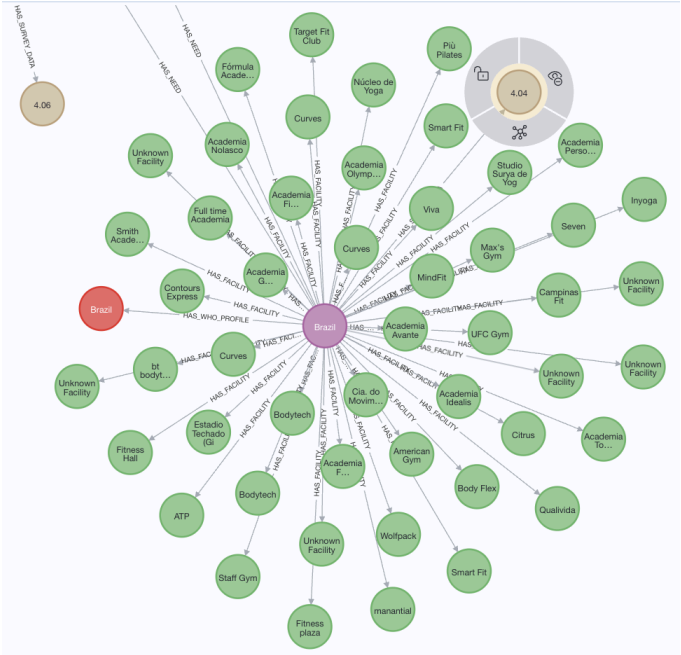
While the HAS_NEED and ADDRESSES edges power the recommendation engine, a Country node is connected to several other node types as well. The screenshot below shows Brazil as the central node with all of its relationships expanded. The large cluster of green Facility nodes connected via HAS_FACILITY edges represents real physical mental health facilities in Brazil pulled from the OSM Overpass API: gyms, yoga studios, fitness centers, and wellness clinics.

The small tan nodes show SurveySnapshot values (composite symptom score of 4.04). The red node is Brazil's WHOProfile, which when selected reveals the WHO data stored for that country: 4.0 psychiatrists per 100,000 people, a mental health budget of 2.3% of total health spending, and a national mental health policy in place. This view illustrates that while the recommendation logic only traverses the HAS_NEED path, the full graph is considerably richer.

```
MATCH (c:Country)-[r]->(n) RETURN c, r, n
```



If we click the tan node, we can take a closer look at the SurveySnapshot node properties for Brazil. These values are the aggregated metrics pulled directly from the PostgreSQL views and stored on the node during the graph loading pipeline. Comparing them against the threshold logic shows exactly which needs were triggered for Brazil: avg_screen_time of 7.05 exceeds the 7.0 threshold activating a Digital Wellness need node, and pct_lacking of 74.7% exceeds the 60% threshold activating an Employer MH Policy need node. Meanwhile avg_stress of 5.53 falls below the 6.0 threshold, avg_symptom_score of 4.04 falls below 5.0, and avg_sleep_hours of 6.78 is above 6.5, so Stress Management, Anxiety & Depression, and Sleep Health need nodes are not triggered for Brazil. This illustrates how the threshold logic produces targeted, country-specific recommendations rather than applying the same needs to every country.



Node properties

SurveySnapshot

<elementId>	4:d7a458e9-31fb-40b9-9efb-f1b4c757f0d2:5
<id>	5
avg_job_satisfaction	5.43
avg_screen_time	7.05
avg_sleep_hours	6.78
avg_stress	5.53
avg_symptom_score	4.04
avg_work_life_balance	5.58
country	Brazil
pct_lacking	74.7

Graph Loading Pipeline

The graph is populated by load_graph.py, which orchestrates ingestion from four sources. The most methodologically significant step is the threshold logic that converts raw PostgreSQL aggregates into MentalHealthNeed nodes.

Threshold Logic

Rather than storing raw numbers directly in the graph, the pipeline applies a set of predefined thresholds to determine which mental health needs are active for a given country. Each triggered threshold creates a HAS_NEED relationship between the country and the corresponding MentalHealthNeed node, with a severity_score property set to the underlying metric value:

Condition (from SQL views)	Need Node Created
avg_stress >= 6.0	Stress Management
avg_symptom_score >= 5.0	Anxiety & Depression
avg_sleep_hours < 6.5	Sleep Health
avg_screen_time >= 7.0	Digital Wellness
pct_lacking_support >= 60%	Employer MH Policy

This threshold design means that the graph only contains NEED nodes that are meaningfully elevated for a country. Therefore, the graph highlights significant issues instead of listing all

possible needs for every country. A country where avg_stress is 5.8 would not generate a Stress Management need node, keeping recommendations targeted rather than generic.

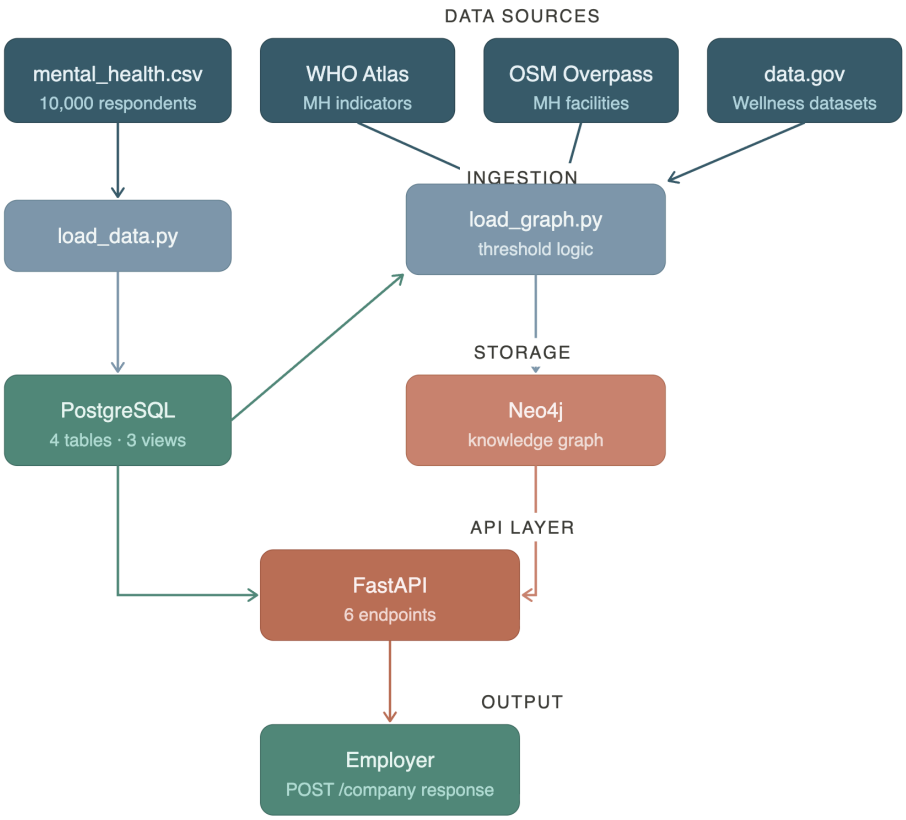
Recommendation Query

The core Cypher query traverses from Country through HAS_NEED to MentalHealthNeed and back through ADDRESSES to WellbeingProgram. Programs are ranked using a coverage_score, which counts how many distinct needs each program addresses for that country. Programs that address more needs receive higher rankings. An optional remote_only filter limits results to programs that support remote participation.

```
cypher = f"""
    MATCH (c:Country {{name: $country}})-[hn:HAS_NEED]->(n:MentalHealthNeed)
          <-[:ADDRESSES]-(p:WellbeingProgram)
    WHERE 1=1 {remote_filter}
    WITH p,
         collect(n.label)            AS addressed_needs,
         collect(hn.severity_score)  AS severity_scores,
         count(n)                    AS coverage_score
    OPTIONAL MATCH (p)-[:EVIDENCED_BY]->(d:GovDataset)
    WITH p, addressed_needs, severity_scores, coverage_score,
         collect(d.title) AS gov_datasets
    RETURN p.program_id    AS program_id,
           p.name          AS name,
           p.program_type  AS program_type,
           p.description   AS description,
           p.remote_ok     AS remote_ok,
           addressed_needs,
           coverage_score,
           gov_datasets
    ORDER BY coverage_score DESC
"""
```

How They All Connect

PostgreSQL and Neo4j are not used in isolation. The graph loading pipeline reads directly from PostgreSQL views to seed the knowledge graph. At query time, the API layer combines results from both databases in a single response, most visibly in the POST /company endpoint, which queries a SQL view for country context and traverses Neo4j for program recommendations in the same request.



FastAPI Layer

The API layer is a FastAPI application exposing six endpoints that bridge PostgreSQL and Neo4j into a single response for the end user.

Endpoint	Method	Description
GET /	GET	Health check
GET /countries	GET	Lists all distinct countries in the respondents table

GET /analysis/{country}	GET	Returns employer gap, lifestyle risk, and treatment gap data from the three SQL views
GET /recommendations/{country}	GET	Graph traversal returning ranked wellbeing programs with optional remote_only filter
GET /graph/{country}	GET	Full graph context: survey snapshot, WHO profile, needs, programs, and facilities
POST /company	POST	Registers a company in Neo4j and returns country context + tailored recommendations

The POST /company endpoint is the core product feature. It calls get_recommendations() internally rather than making an HTTP round-trip, avoiding unnecessary network overhead. The remote_policy field in the request body controls whether the Cypher query filters to remote_ok programs only.

Case Studies

Case 1: TechCorp Munich (Germany, Hybrid, Medium, Technology)

A mid-sized technology company in Munich submits a POST /company request with remote_policy: hybrid. The system queries employer_gap_by_country and returns Germany's context: avg_stress 5.60/10 (highest of all countries), pct_lacking_support 76.8% (also highest), and avg_symptom_score 4.03/10. The graph traversal returns three recommended programs:

- Peer Support & MH Champions
- MH Policy & Manager Training
- Employee Assistance Program (EAP)

① API REQUEST POST /COMPANY

```python
# api/main.py → @app.post("/company")
# Request body
{
  "name":    "TechCorp Munich",
  "country": "Germany",
  "industry":"Technology",
  "size":    "medium",
  "remote_policy": "hybrid"
}

# Handler logic
async def register_company(body):
  # 1. Insert into PostgreSQL
  company_id = pg_query(
    "INSERT INTO companies ..."
  )
  # 2. Query SQL view
  context = pg_query(
    employer_gap_by_country,
    country=body.country
  )
  # 3. Traverse graph
  recs = neo4j_query(
    cypher, country=body.country,
    remote_only=False
  )
```

② SQL VIEW QUERY POSTGRESQL

```sql
-- sql/schema.sql → employer_gap_by_country
CREATE VIEW employer_gap_by_country AS
SELECT
  r.country,
  AVG(l.stress_level)
    AS avg_stress,
  COUNT(*) FILTER (
    WHERE NOT
      w.company_mental_health_support
  ) * 100.0 / COUNT(*)
    AS pct_lacking_support,
  AVG(m.composite_symptom_score)
    AS avg_symptom_score
FROM respondents r
JOIN work_profiles w    USING (respondent_id)
JOIN lifestyle_factors l USING (respondent_id)
JOIN mental_health_status m USING (respondent_id)
GROUP BY r.country;
```

RESULT — GERMANY (968 RESPONDENTS)

avg_stress	5.60 / 10 ↑ highest
pct_lacking_support	76.8% ↑ highest
avg_symptom_score	4.03 / 10
treatment_gap	59.3% untreated

③ GRAPH TRAVERSAL NEO4J CYPHER

```cypher
// api/main.py → neo4j_query()
MATCH
  (c:Country {name: "Germany"})
  -[:HAS_NEED]->
  (n:MentalHealthNeed)
  <-[:ADDRESSES]-
  (p:WellbeingProgram)
WHERE p.remote_ok = true
RETURN
  p.program_id,
  p.name,
  p.program_type,
  n.need_type
    AS addressed_need
ORDER BY
  p.coverage_score DESC
```

PEER_001	Peer Support & MH Champions	peer_support
POLICY_001	MH Policy & Manager Training	employer_policy
EAP_001	Employee Assistance Program	EAP

REQUEST FLOW POST /company → INSERT companies (PG) → pg_query(employer_gap_by_country) → neo4j_query(MATCH c:Country → n:Need ← p:Program) → 3 recommendations returned

Case 2: São Paulo Startup (Brazil, Remote Only, Small, Technology)

A small remote-first startup in São Paulo submits the same request with remote_policy: yes. This activates the remote_only=True filter in the Cypher query. Brazil's context shows avg_stress 5.53/10 and pct_lacking_support 74.7%. Notably, Brazil has only 4 psychiatrists per 100k population versus Germany's 14, and a mental health budget of 2.3% versus Germany's 11.3%. We see the same survey patterns but far weaker national infrastructure. The graph traversal returns four programs (one more than Germany due to Brazil's digital_wellness need node being active):

- Peer Support & MH Champions
- MH Policy & Manager Training
- Employee Assistance Program (EAP)
- Digital Wellness & Screen Time (remote_ok = true -> new for this case)

① API REQUEST POST /COMPANY

```
api/main.py → @app.post("/company")

# Request body
{
  "name":    "StartupSP",
  "country": "Brazil",
  "industry":"Technology",
  "size":    "small",
  "remote_policy": "yes"  ← key diff

}

# Same handler, different outcome:
# remote_policy:"yes" passes
# remote_only=True to neo4j_query
# → filters to remote_ok programs only
# → digital_wellness need active
# → DIGWEL_001 added to results

  recs = neo4j_query(
    cypher, country="Brazil",
    remote_only=True  ← activated

  )
```

② SQL VIEW RESULT POSTGRESQL

```
sql/schema.sql → treatment_gap_by_country

CREATE VIEW treatment_gap_by_country AS
SELECT
  r.country,
  COUNT(*) FILTER (
    WHERE m.has_mental_health_diagnosis
  ) AS diagnosed,
  COUNT(*) FILTER (
    WHERE m.has_mental_health_diagnosis
    AND NOT m.currently_in_treatment
  ) AS untreated,
  untreated * 100.0
    / NULLIF(diagnosed, 0)
    AS treatment_gap_pct
FROM respondents r
JOIN mental_health_status m
```

RESULT — BRAZIL (1,077 RESPONDENTS)

avg_stress	5.53 / 10
pct_lacking_support	74.7%
avg_symptom_score	4.04 / 10
treatment_gap	60.4% untreated

WHO: 4 psychiatrists/100k vs Germany's 14. MH budget 2.3% vs 11.3%. Same survey patterns, far weaker national infrastructure.

③ GRAPH TRAVERSAL NEO4J CYPHER

```
api/main.py → neo4j_query(remote_only=True)

MATCH
  (c:Country {name: "Brazil"})
  -[:HAS_NEED]->
  (n:MentalHealthNeed)
  <-[:ADDRESSES]-
  (p:WellbeingProgram)
 WHERE p.remote_ok = true  ← filtered

RETURN
  p.program_id,
  p.name,
  p.program_type,
  n.need_type
    AS addressed_need
```

PEER_001	Peer Support & MH Champions	peer_support
POLICY_001	MH Policy & Manager Training	employer_policy
EAP_001	Employee Assistance Program	EAP
DIGWEL_001	Digital Wellness & Screen Time	digital_wellness +1 NEW

KEY DIFFERENCE remote_policy:"yes" → remote_only=True passed to Cypher WHERE clause → digital_wellness need node active in Brazil graph → 4 recommendations (not 3)

The key difference between the two cases is that remote_policy: yes passes remote_only=True to the Cypher WHERE clause, which filters out any program where remote_ok = false. Brazil's digital_wellness need node is also active (avg_screen_time threshold met), which adds the Digital Wellness program to the result set, bringing the total to 4 recommendations versus Germany's 3.

Design Considerations

Why Two Databases?

PostgreSQL is suited to the analytical workload: computing averages, percentages, and ranked aggregates across 10,000 structured rows with well-defined schemas. SQL views allow these aggregations to be pre-computed and accessed quickly by the API.

Neo4j is suited to the recommendation workload: finding all programs that address a set of needs linked to a country is fundamentally a graph traversal. Modeling this logic in SQL would require multiple join tables and complex queries, while graph relationships such as ADDRESSES and EVIDENCED_BY make these connections more intuitive.

The two databases complement rather than duplicate each other. PostgreSQL stores the raw survey data and statistical summaries, while Neo4j stores the domain knowledge and relationship structure. The graph loading pipeline bridges them by translating SQL aggregates into graph properties, so the graph is always derived from the ground-truth relational data.

Schema Normalization

The four-table schema avoids wide-table issues by separating the four domains into independent tables, each with a one-to-one relationship to respondents. The composite_symptom_score computed column is a deliberate denormalization: it eliminates repeated arithmetic at query time and avoids inconsistent re-implementation of the formula across multiple queries.

Graph Constraints and Re-runability

All seven node types have uniqueness constraints defined in constraints.cypher. All node creation uses MERGE rather than CREATE, so the pipeline can be re-run safely and existing nodes are updated in place rather than duplicated. This matters because the OSM and data.gov sources may return different results on different runs as those external datasets are updated.

Connection Management

Both database connections are created once when first needed and reused for the lifetime of the application. The PostgreSQL connection is validated on each use by executing SELECT 1 and re-established if closed. Neo4j uses a persistent driver instance. Both are explicitly closed in FastAPI's lifespan handler to prevent connection leaks on shutdown.

Limitations and Future Considerations

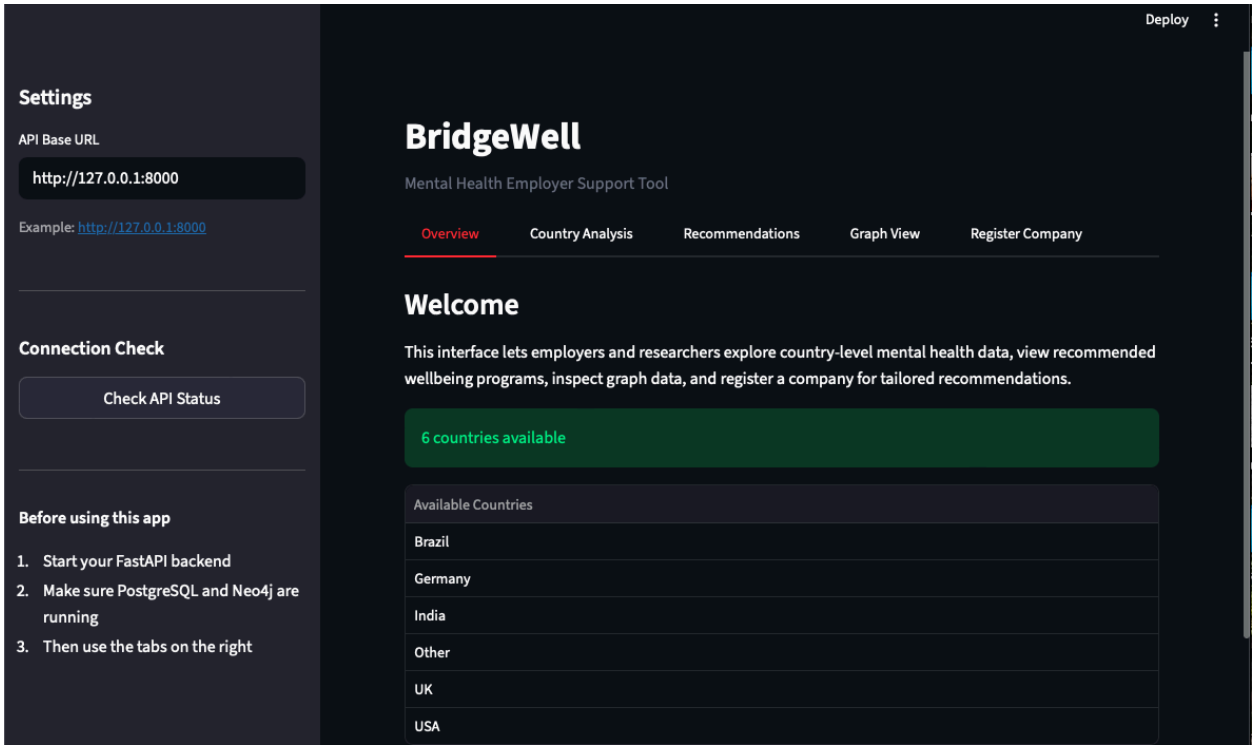
The main limitation of BridgeWell is the survey dataset. While 10,000 respondents gives us a solid starting point, the data is not evenly spread across countries. Some countries have very few respondents, which makes their aggregates less reliable, and some smaller countries don't appear in the dataset at all.

The wellbeing programs in the graph were also added manually. Ideally these would come from a larger, expert-curated database with more detail on each program, such as cost, language, and whether it's been formally accredited.

For future work, we'd like to incorporate WHO and facility data more directly into the recommendation logic rather than just storing it as context. We'd also want to expand the dataset to cover more countries more evenly, and potentially add a way for companies to rate programs so recommendations improve over time.

Streamlit Frontend

In addition to the API, we built a simple Streamlit frontend that provides a browser-based interface for interacting with BridgeWell. It connects to the FastAPI backend and lets users explore country analysis, view recommendations, inspect graph data, and register a company through a form without needing to make raw API calls.



Conclusion

BridgeWell shows how using two databases together can power a smarter recommendation system than either one could alone. PostgreSQL handles the heavy lifting of aggregating survey data, while Neo4j lets us traverse relationships between countries, needs, and programs to surface relevant recommendations.

The threshold logic that converts SQL averages into graph need nodes is central to the system's design. It ensures that recommendations reflect the mental health landscape of each country rather than returning the same programs for everyone. The two case studies show this in action, where the same endpoint returns different results depending on the country's data and whether the company is remote.